Programming Assignment 2

EC5504 Computational Photography, Fall 2026
Prof. Jae Woong Soh

Due Friday, Nov. 13, 2026, at 11:59 PM KST

1. Image Blending

In this assignment, you will implement image blending using Laplacian pyramids. The objective is to understand multi-scale image representations and how they can be used to blend images seamlessly. You will seamlessly merge two images, creating a smooth transition between them. For this task, you need two input images and a binary mask. An example blended image of an apple and an orange is shown in Figure 1.



Figure 1: A seamlessly blended image of an apple and an orange.

1.1. Gaussian and Laplacian Pyramids

Implement `gaussian_pyramid` and `laplacian_pyramid`, which generate 7-level Gaussian and Laplacian pyramids, respectively. Reconstruct the original image by collapsing the Laplacian pyramid, and verify the reconstruction accuracy. You may use `NumPy` and basic filtering operations, but pyramid construction and reconstruction must be implemented by you.

1.2. Multi-band Image Blending

Given two images I_A and I_B, and a blending mask $M \in [0, 1]$, implement multi-band image blending.

Using the Gaussian and Laplacian pyramids, compute the blended Laplacian pyramid using the following equation:

$$\mathbf{L}_k = \mathbf{L}_k^A \cdot \mathbf{m}_k + \mathbf{L}_k^B \cdot (1 - \mathbf{m}_k),$$

where $\mathbf{L}_k$ denotes the Laplacian pyramid at level k, and $\mathbf{m}_k$ represents the Gaussian pyramid for the mask corresponding to the blending region at the k-th level.

Finally, collapse the blended Laplacian pyramid to reconstruct the final blended image.

Question: How does changing the number of pyramid levels affect the blending result? How does changing the Gaussian kernel size affect the blending result?

2. Image Enhancement

In this problem, you will implement several algorithms for *Image Enhancement*. You may use your own images.

Part 1. Detail Enhancement: Unsharp Masking

You will implement an *image sharpening* algorithm to enhance the original image. Unsharp masking enhances image details by adding a scaled high-frequency component to the original image.

Read an image and implement the unsharp masking algorithm:

$$I_{out} = I_{in} + \alpha\big(I_{in} - G_\sigma * I_{in}\big).$$

Analyze the result with respect to the parameters you choose. Instead of strictly following the mathematical expression, you may implement the method using a single convolution with a predefined kernel, as discussed in class.

Question: What are the respective effects of α and σ on the sharpening result?

Part 2. Contrast Enhancement: Histogram Equalization

Histogram equalization is an intensity transformation technique that redistributes image intensities based on the cumulative distribution function (CDF). For this problem, you may use your own images, or the examples that are provided with the assignment.

Read an image and implement **histogram equalization**. Compute the histogram $h(r_k)$ of intensity levels r_k, and the corresponding cumulative histogram $c(r_k)$:

$$c(r_k) = \sum_{j=0}^{k} h(r_j)$$

Derive the CDF to define an intensity mapping function for intensity transformation:

$$s_k = \frac{c(r_k)}{MN}$$

where MN is the total number of pixels.

Apply this mapping to obtain the equalized image. Apply histogram equalization to color images using a luminance-based approach (`YCbCr` color space). Analyze the resulting image and its histogram, and find failure cases (e.g., images with a bimodal histogram where global equalization may over-enhance certain regions).

3. HDRI and Tone Mapping

In this problem, you will explore high dynamic range (HDR) imaging, radiometric calibration, and tone mapping. As discussed in class, HDR imaging enables the creation of floating-point precision images that linearly correspond to scene radiance values. Tone mapping compresses the dynamic range of HDR images into an 8-bit range, so that they can be shown on a display. You will use a set of exposure brackets that are provided with the assignment.

(Assume that the images are captured with fixed aperture and ISO, and with shutter speeds equal to $\frac{1}{2^{k-1}}$, where $k \in \{1, 2, \cdots\}$ is the index of an image, unless otherwise provided.) You may use OpenCV to read and save HDR images in Radiance `.hdr` format.

Part 1. Radiometric Calibration: Linearize Bracketed Images

The common LDR images are non-linear. Before merging them into an HDR image, you first need to perform radiometric calibration to estimate Camera Response Functions (CRFs), and then linearize the images. You will follow the method by Debevec and Malik.

An intensity $I_{ij} \in \{0, \cdots, 255\}$ at pixel i of exposure j is related to scene radiance E_i:

$$I_{ij} = f(E_i \Delta t_j),$$

where Δt_j is the exposure time of image j, and $f(\cdot)$ is the unknown non-linear CRF. Once we obtain $f^{-1}(\cdot)$, we can invert I_{ij} back to linear measurements.

As mentioned in class, instead of estimating f^{-1}, we estimate $g = \log f^{-1}$, where

$$g(I_{ij}) = \log(E_i) + \log(\Delta t_j).$$

The domain of g is the set of discrete intensity values $\{0, \cdots, 255\}$, essentially working as a 256-dimensional lookup table.

The unknowns are g and E_is. We use the fact that irradiance E_i is constant across all LDR images. The optimization problem is given by:

$$\min_{g, E_i} \sum_{i,j} [w(I_{ij}/255)(g(I_{ij}) - \log(E_i) - \log(\Delta t_j))]^2 + \lambda \sum_{z=1}^{254} [w(z/255)\nabla^2 g(z)]^2$$

Note that $\nabla^2 g(z) = g(z+1) - 2g(z) + g(z-1)$, and $w(z)$ is a weighting function. The parameter λ controls the strength of regularization, which affects the final output. (Use the **tent weighting function** defined in Part 2. for CRF estimation.)

Constructing the Least-Squares Problem

Express the problem as a least-squares optimization problem by expressing it in matrix form:

$$\|A\mathbf{v} - \mathbf{b}\|^2,$$

where A is a matrix and $v = [g; \log E_i]$ are the unknowns, and b is a known vector.

1.1. Define the unknown vector

$$\mathbf{v} = [g(0), g(1), \ldots, g(255), \log E_1, \ldots, \log E_N]^\top$$

The first 256 elements represent the camera response function, while the remaining N elements represent the log irradiances at the sampled pixels.

1.2. Construct the data-fitting equations

For each sampled pixel i and exposure image j, the imaging model gives

$$w_{ij} g(I_{ij}) - w_{ij} \log E_i = w_{ij} \log \Delta t_j, \quad \text{where } w_{ij} = w(I_{ij}/255).$$

Each pixel-exposure pair contributes one row to A and one element to $\mathbf{b}$. The corresponding row contains only two nonzero coefficients:

- $+w_{ij}$ at the column corresponding to $g(I_{ij})$.
- $-w_{ij}$ at the column corresponding to $\log E_i$.

The corresponding element of $\mathbf{b}$ is $w_{ij} \log \Delta t_j$.

1.3. Add the scale constraint

The response function and irradiance values can only be determined up to a common scale factor. To remove this ambiguity, impose the constraint

$$g(128) = 0.$$

Add one row to A with a value of 1 at the column corresponding to $g(128)$ and zeros elsewhere. The corresponding element of $\mathbf{b}$ is zero.

1.4. Add the smoothness equations

To encourage a smooth camera response function, add one equation for each $z = 1, \ldots, 254$:

$$\sqrt{\lambda} w(z/255)[g(z-1) - 2g(z) + g(z+1)] = 0.$$

Each smoothness equation contributes three nonzero coefficients to the corresponding row of A. Note that $\sqrt{\lambda}$, rather than λ, is used to match the squared regularization term in the objective function ($\sqrt{\lambda} \approx 10{-}100$).

1.5. Solve the problem

The resulting matrix and vectors have dimensions

$$A \in \mathbb{R}^{(NP+255)\times(256+N)}, \quad \mathbf{v} \in \mathbb{R}^{256+N}, \quad \mathbf{b} \in \mathbb{R}^{NP+255}.$$

You may use a sparse matrix representation if necessary to reduce memory consumption.

It is recommended to use around $500 - 1000$ randomly sampled pixels. Estimate a separate camera response function for each of the R, G, and B channels. You may use a numerical linear algebra solver, such as `numpy.linalg.lstsq`, but must construct the optimization system yourself.
- Convert the non-linear images into linear images $I_{ij,\text{lin}} = \exp(g(I_{ij}))$.
- Plot the function g and f that you derived.
- Investigate the effect of the regularization parameter λ.

Part 2. Merge Exposure Brackets into HDR Image

Now merge the set of linear images:

$$I_{i,\text{HDR}} = \exp\left(\frac{\sum_j w(I_{ij,LDR})(\log(I_{ij,\text{lin}} + \epsilon) - \log(\Delta t_j))}{\sum_j w(I_{ij,LDR})}\right).$$

- Weighting functions

$$w_{\text{unif}}(z) = \begin{cases} 1, & \text{if } Z_{min} \le z \le Z_{max}, \\ 0, & \text{otherwise} \end{cases},$$

$$w_{\text{tent}}(z) = \begin{cases} \min(z, 1-z), & \text{if } Z_{min} \le z \le Z_{max}, \\ 0, & \text{otherwise} \end{cases},$$

$$w_{\text{Gaussian}}(z) = \begin{cases} \exp\left(-4\frac{(z-0.5)^2}{0.5^2}\right), & \text{if } Z_{min} \le z \le Z_{max}, \\ 0, & \text{otherwise} \end{cases}$$

All weights assume the intensity values $z \in [0, 1]$. Use $Z_{min} = 0.05,\ Z_{max} = 0.95$ as recommended.
Choose one weighting method that you find the most visually pleasing. (Save as an HDR file format).
Note: Handle pixels for which the sum of weights is zero.

Part 3. Tone Mapping

HDR radiance maps cannot be directly represented on conventional low dynamic range displays without dynamic range compression. Now you will implement several global tone mapping algorithms.

(a) Reinhard Tone Mapping: refer to the lecture slides, set $L_{\text{white}} = B \cdot \max(L)$.

$$L_d = \frac{L}{1+L}(1 + L/L_{\text{white}}^2)$$

(b) Logarithmic Tone Mapping.

$$L_d = \frac{\log(1+\beta L)}{\log(1+\beta L_{\max})}, \qquad L_{\max} = \max(L).$$

Here, L denotes the scaled luminance, as discussed in class. Experiment with different values of B and β.

(c) Additionally, tone map the provided "HDR image" and repeat (a-b).

Finally, discuss your results and explain which tone mapping method you prefer. Compare the tone-mapped image with the original input exposure images.

Questions: What information is lost or compressed during tone mapping?



LDR Image



HDR Tone mapped Image

`MMSPG_0026.hdr`: LDR/HDR image pair.

Submission Requirements

- Source code
- Output and intermediate results
- Report including:
  - Explanation of your implementation
  - Output and intermediate results
  - Analyses and discussions
  - Answers to all questions

Academic Integrity and Use of AI Tools

You are expected to understand and be able to explain all submitted code. The core algorithms required in this assignment must be implemented by you.
If generative AI tools are used for auxiliary purposes, their use must be clearly disclosed in the report. Generative AI must not be used to generate complete assignment solutions.